

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ОДЕСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ імені І. І. МЕЧНИКОВА
ФАКУЛЬТЕТ МАТЕМАТИКИ, ФІЗИКИ ТА ІНФОРМАЦІЙНИХ
ТЕХНОЛОГІЙ
КАФЕДРА МАТЕМАТИЧНОГО ЗАБЕЗПЕЧЕННЯ КОМП'ЮТЕРНИХ
СИСТЕМ

ЗВІТ

до практичного завдання
з дисципліни «Мультиагентні системи»

за темою:

«Реалізація та аналіз стратегічних ігор у мультиагентних системах»

Виконав:

студент 4 курсу

спеціальності 126

Владислав Краковський

Перевірив:

Пенко В. Г.

Одеса - 2026

Зміст

1. ВСТУП.....	3
2. МЕТА ТА ПОСТАНОВКА ЗАДАЧІ.....	4
3. ТЕОРЕТИЧНІ ВІДОМОСТІ.....	4
3.1. Поняття мультиагентної системи.....	4
3.2. Стратегічна гра у нормальній формі.....	5
3.3. Ітеративне вилучення строго домінованих стратегій.....	5
3.4. Рівновага Неша.....	5
4. МАТЕМАТИЧНА МОДЕЛЬ СТРАТЕГІЧНИХ ІГОР.....	6
4.1. Дилема ув'язненого.....	6
4.2. Гра в монетку.....	6
4.3. Гра на перехресті.....	7
5. ОПИС ПРОГРАМНОЇ РЕАЛІЗАЦІЇ.....	7
5.1. Структура Game.....	7
5.2. Основні функції програми.....	8
5.3. Алгоритм пошуку домінованих стратегій.....	8
5.4. Алгоритм пошуку рівноваг Неша.....	8
6. РЕЗУЛЬТАТИ ВИКОНАННЯ ПРОГРАМИ.....	8
7. ТЕСТУВАННЯ ПРОГРАМИ.....	11
8. ІНСТРУКЦІЯ КОРИСТУВАЧА.....	11
9. ВИСНОВКИ.....	12
ДОДАТОК А. ТЕКСТ ПРОГРАМИ.....	13

1. ВСТУП

Мультіагентні системи є одним із напрямів штучного інтелекту, у якому вивчається взаємодія кількох автономних агентів. Кожний агент має власні дії, власну інформацію та власний критерій корисності. Через це результат системи залежить не тільки від рішення окремого агента, а й від рішень інших учасників взаємодії.

У цій роботі розглянуто три класичні стратегічні ігри: дилему ув'язненого, гру в монетку та гру на перехресті. Вони дозволяють показати різні типи взаємодії агентів: конфлікт інтересів, строго конкурентну гру та координаційну ситуацію. На їх прикладі реалізовано пошук строго домінованих стратегій і чистих рівноваг Неша.

Програмна реалізація виконана мовою C++. У програмі використано єдину структуру даних для опису гри, а також універсальні функції для виведення матриці виграшів, аналізу IESDA та пошуку рівноваг Неша. Назви стратегій і текст консольного виведення в програмі подано російською мовою згідно з вимогою до реалізації.

2. МЕТА ТА ПОСТАНОВКА ЗАДАЧІ

Мета роботи - розробити консольну програму мовою C++, яка моделює та аналізує три стратегічні ігри у нормальній формі з погляду мультиагентних систем.

Для досягнення мети необхідно виконати такі завдання:

- описати три стратегічні ігри за допомогою матриць виграшів;
- реалізувати збереження назв ігор, описів ситуацій, стратегій гравців і матриць виграшів;
- реалізувати пошук строго домінованих стратегій для кожного гравця;
- реалізувати пошук чистих рівноваг Неша шляхом перебору всіх клітин матриці;
- вивести результати аналізу для кожної гри;
- підготувати висновки щодо поведінки агентів у різних типах стратегічних взаємодій.

Об'єктом дослідження є стратегічна взаємодія двох агентів. Предметом дослідження є алгоритмічне визначення домінованих стратегій і чистих рівноваг Неша в іграх з двома гравцями.

3. ТЕОРЕТИЧНІ ВІДОМОСТІ

3.1. Поняття мультиагентної системи

Мультиагентна система складається з кількох автономних агентів, які можуть одночасно діяти в одному середовищі. Кожен агент сприймає стан середовища, обирає дії та отримує результат відповідно до своєї цільової функції. На відміну від одноагентної системи, тут поведінка одного агента впливає на можливості та виграші інших агентів.

У стратегічних задачах агенти часто розглядаються як раціональні учасники, які прагнуть максимізувати власний виграш. Таке припущення дозволяє застосовувати апарат теорії ігор для формального аналізу взаємодії.

3.2. Стратегічна гра у нормальній формі

Стратегічна гра у нормальній формі описується множиною гравців, множиною стратегій кожного гравця та функціями виграшу. Для двох гравців така гра може бути подана у вигляді матриці, де рядок відповідає стратегії першого гравця, стовпець - стратегії другого гравця, а клітина матриці містить пару виграшів.

У загальному вигляді елемент матриці можна записати так:

$$a_{ij} = (u_1(s_i, t_j), u_2(s_i, t_j))$$

де s_i - стратегія першого гравця, t_j - стратегія другого гравця, u_1 та u_2 - функції виграшу відповідно першого і другого гравців.

3.3. Ітеративне видалення строго домінованих стратегій

Стратегія називається строго домінованою, якщо існує інша стратегія того самого гравця, яка дає йому строго більший виграш за будь-якої стратегії іншого гравця. Метод IESDA базується на припущенні, що раціональний агент не буде обирати стратегію, яка завжди гірша за іншу доступну стратегію.

$$u_i(a'_i, a_{-i}) > u_i(a_i, a_{-i}), \text{ для всіх } a_{-i}$$

У програмі ця перевірка реалізована через порівняння рядків матриці для першого гравця та стовпців матриці для другого гравця.

3.4. Рівновага Неша

Рівновага Неша - це такий набір стратегій, за якого жоден гравець не може покращити свій виграш односторонньою зміною власної стратегії. Іншими словами, кожна стратегія у профілі є найкращою відповіддю на стратегії інших гравців.

$$u_i(a_i^*, a_{-i}^*) \geq u_i(a_i, a_{-i}^*), \text{ для всіх } a_i$$

У програмі кожна клітина матриці перевіряється як потенційна рівновага. Для цього програма аналізує, чи може перший гравець отримати більший виграш, змінивши тільки рядок, і чи може другий гравець отримати більший виграш, змінивши тільки стовпець.

4. МАТЕМАТИЧНА МОДЕЛЬ СТРАТЕГІЧНИХ ІГОР

4.1. Дилема ув'язненого

У дилемі ув'язненого два агенти незалежно обирають одну з двох стратегій: мовчати або зізнатися. Якщо обидва мовчать, обидва отримують добрий результат. Якщо один зізнається, а інший мовчить, той, хто зізнався, отримує найкращий результат, а інший - найгірший. Якщо обидва зізнаються, обидва отримують посередній результат.

Таблиця 4.1 - Матриця виграшів для дилеми ув'язненого

Ігрок 1 / Ігрок 2	Мовчати	Зізнатися
Мовчати	(3; 3)	(0; 4)
Зізнатися	(4; 0)	(1; 1)

4.2. Гра в монетку

Гра в монетку є прикладом строго конкурентної гри. Перший гравець виграє, якщо вибори сторін монети збігаються. Другий гравець виграє, якщо вибори різні. Сума виграшів у кожній клітині дорівнює нулю, тому виграш одного гравця є програшем іншого.

Таблиця 4.2 - Матриця виграшів для гри в монетку

Ігрок 1 / Ігрок 2	Орел	Решка
Орел	(1; -1)	(-1; 1)
Решка	(-1; 1)	(1; -1)

4.3. Гра на перехресті

Гра на перехресті є координаційною ситуацією. Два водії повинні вирішити, їхати чи зупинитися. Якщо обидва їдуть, виникає конфлікт. Якщо один їде, а інший зупиняється, виграє той, хто проїхав. Якщо обидва зупиняються, ніхто не отримує додаткового виграшу.

Таблиця 4.3 - Матриця виграшів для гри на перехресті

Ігрок 1 / Ігрок 2	Їхати	Зупинитися
Їхати	(-1; -1)	(1; 0)
Зупинитися	(0; 1)	(0; 0)

5. ОПИС ПРОГРАМНОЇ РЕАЛІЗАЦІЇ

Програма реалізована мовою C++ як один файл. Такий підхід дозволяє уникнути дублювання коду: функції аналізу описані один раз і застосовуються до трьох різних ігор. Це практичніше, ніж створювати три окремі програми з однаковими циклами.

5.1. Структура Game

Для збереження даних гри використано структуру Game. Вона містить назву гри, опис ситуації, список стратегій першого гравця, список стратегій другого гравця та матрицю виграшів. Один елемент матриці має тип Payoff, тобто пару цілих чисел.

5.2. Основні функції програми

- `printMatrix` - виводить назву гри, опис ситуації та матрицю виграшів;
- `findDominatedStrategies` - порівнює стратегії гравців і визначає строго доміновані стратегії;
- `findPureNashEquilibria` - перебирає всі клітини матриці та перевіряє умову рівноваги Неша;
- `analyzeGame` - виконує повний аналіз однієї гри: вивід матриці, IESDA та пошук рівноваг Неша.

5.3. Алгоритм пошуку домінованих стратегій

Для першого гравця програма порівнює кожен рядок. Якщо один рядок дає строго більший виграш за всіма стовпцями, інший рядок вважається строго домінованим. Для другого гравця аналогічно порівнюються стовпці за другим компонентом виграшу.

5.4. Алгоритм пошуку рівноваг Неша

Кожна клітина матриці розглядається як кандидат на рівновагу. Для неї перевіряється, чи є поточний рядок найкращою відповіддю першого гравця на вибраний стовпець, а поточний стовпець - найкращою відповіддю другого гравця на вибраний рядок. Якщо обидві умови виконуються, клітина виводиться як чиста рівновага Неша.

6. РЕЗУЛЬТАТИ ВИКОНАННЯ ПРОГРАМИ

Таблиця 4.1 - Підсумкові результати аналізу стратегічних ігор

Гра	Результат IESDA	Чисті рівноваги Неша
Дилема ув'язненого	Стратегія «Мовчати» строго домінується	(Зізнатися; Зізнатися), виграш (1; 1).

	стратегією «Зізнатися» для обох гравців.	
Гра в монетку	Строго домінованих стратегій немає.	Чистої рівноваги Неша немає. Змішана рівновага: 0,5 / 0,5.
Гра на перехресті	Строго домінованих стратегій немає.	(Їхати; Зупинитися), виграш (1; 0); (Зупинитися; Їхати), виграш (0; 1).

Результати показують, що для дилеми ув'язненого існує одна домінантна поведінка: обидва агенти обирають зізнання. У грі в монетку чистої рівноваги немає, оскільки кожен гравець має стимул змінити вибір. У грі на перехресті є дві чисті рівноваги, які відповідають узгодженій поведінці: один агент їде, інший зупиняється.

Рисунок 6.1 - Фрагмент консольного вывода программы

```
=====
Дилемма заключённого
=====

Ситуация игры:
Два заключённых выбирают: молчать или признаться. Каждый принимает решение независимо. Если один признаётся, а другой молчит, признавшийся получает лучший результат.

Формат выигрыша: (Игрок 1, Игрок 2)

      Молчать      Признаться
Молчать      (3, 3)      (0, 4)
Признаться    (4, 0)      (1, 1)

Анализ IESDA:
Игрок 1: стратегия "Молчать" строго доминируется стратегией "Признаться".
Игрок 2: стратегия "Молчать" строго доминируется стратегией "Признаться".

Чистые равновесия Нэша:
(Признаться, Признаться) выигрыш = (1, 1)

=====
Игра в монетку
=====

Ситуация игры:
Два игрока одновременно выбирают сторону монеты: орёл или решка. Первый игрок выигрывает, если выборы совпали. Второй игрок выигрывает, если выборы различаются.

Формат выигрыша: (Игрок 1, Игрок 2)

      Орёл      Решка
Орёл      (1, -1)      (-1, 1)
Решка      (-1, 1)      (1, -1)

Анализ IESDA:
Строго доминируемых стратегий не найдено.

Чистые равновесия Нэша:
Чистое равновесие Нэша не найдено.
Смешанное равновесие: каждый игрок выбирает "Орёл" и "Решка" с вероятностью 0.5.

=====
Игра на перекрёстке
=====

Ситуация игры:
Два водителя подъезжают к перекрёстку. Каждый может ехать или остановиться. Если оба едут, возникает конфликт. Если один едет, а другой стоит, выигрывает тот, кто проехал.

Формат выигрыша: (Игрок 1, Игрок 2)

      Ехать      Остановиться
Ехать      (-1, -1)      (1, 0)
Остановиться    (0, 1)      (0, 0)

Анализ IESDA:
Строго доминируемых стратегий не найдено.

Чистые равновесия Нэша:
(Ехать, Остановиться) выигрыш = (1, 0)
(Остановиться, Ехать) выигрыш = (0, 1)
```

7. ТЕСТУВАННЯ ПРОГРАМИ

Для перевірки коректності програми було виконано запуск з трьома заздалегідь заданими матрицями виграшів. Очікувані результати порівнювалися з теоретичним аналізом ігор.

У дилемі ув'язненого стратегія «Мовчати» є строго домінованою для обох гравців, тому очікуваний результат IESDA - пара стратегій «Зізнатися, Зізнатися». Програма виводить саме цей результат.

У грі в монетку жодна стратегія не є строго домінованою, а чистої рівноваги Неша немає. Програма також додатково повідомляє про відому змішану рівновагу, у якій кожен гравець обирає кожную сторону монети з імовірністю 0,5.

У грі на перехресті програма знаходить дві рівноваги Неша: «Їхати, Зупинитися» та «Зупинитися, Їхати». Це відповідає змісту задачі: конфлікту немає тоді, коли один учасник проїжджає, а інший поступається.

8. ІНСТРУКЦІЯ КОРИСТУВАЧА

Для роботи з програмою потрібно мати встановлений компілятор C++, наприклад g++ з пакета MinGW, MSYS2 або GNU Compiler Collection у Linux.

Компіляція програми виконується командою:

```
g++ multiagent_games.cpp -o multiagent_games
```

Запуск у Linux, MSYS2 або Git Bash:

```
./multiagent_games
```

Запуск у Windows CMD або PowerShell:

```
multiagent_games.exe
```

Після запуску програма автоматично аналізує всі три ігри та виводить у консоль матриці виграшів, результати IESDA і знайдені рівноваги Неша.

9. ВИСНОВКИ

У ході роботи було реалізовано консольну програму мовою C++, яка аналізує три стратегічні ігри у нормальній формі. Програма не обмежується простим виведенням готового тексту, а виконує перевірку домінованих стратегій і пошук чистих рівноваг Неша за матрицями виграшів.

Результати демонструють різні типи мультиагентної взаємодії. У дилемі ув'язненого індивідуально раціональна поведінка приводить до не найкращого спільного результату. У грі в монетку відсутня чиста рівновага, тому стабільна поведінка описується змішаними стратегіями. У грі на перехресті рівновага виникає за умови координації дій агентів.

Розроблена програма має просту структуру, але є достатньо універсальною для аналізу двоособових ігор із довільними назвами стратегій і матрицями виграшів. Отже, поставлена мета роботи виконана.

ДОДАТОК А. ТЕКСТ ПРОГРАМИ

Нижче наведено повний текст програми мовою C++, використаний для аналізу трьох стратегічних ігор.

```
#include <iostream>          // Бібліотека для вивода тексту в консоль
#include <vector>             // Бібліотека для роботи з vector
#include <string>             // Бібліотека для роботи со строками
#include <utility>            // Бібліотека для pair
#include <locale>             // Бібліотека для настройки локалі

#ifdef _WIN32
#include <windows.h>          // Нужно для нормального вывода кириллицы в
Windows
#endif

using namespace std;

// Выигрыш хранит результат двух игроков.
// first - выигрыш первого игрока.
// second - выигрыш второго игрока.
using Payoff = pair<int, int>;

// Структура Game хранит данные одной игры.
struct Game {
    string name;              // Название игры
    string situation;         // Краткое описание ситуации
игры
    vector<string> rowActions; // Стратегии первого игрока,
строки матрицы
    vector<string> colActions; // Стратегии второго игрока,
столбцы матрицы
    vector<vector<Payoff>> matrix; // Матрица выигрышей
};

// Функция выводит матрицу выигрышей.
void printMatrix(const Game& game) {
    cout << "\n===== \n";
    cout << game.name << endl;
    cout << "===== \n\n";

    // Выводим описание ситуации игры.
    cout << "Ситуация игры: \n";
    cout << game.situation << "\n\n";

    // Поясняем формат записи выигрышей.
    cout << "Формат выигрыша: (Игрок 1, Игрок 2) \n\n";

    // Выводим стратегии второго игрока.
    cout << " ";
    for (const string& colName : game.colActions) {
```

```

        cout << colName << "          ";
    }
    cout << endl;

    // Выводим строки матрицы.
    for (int i = 0; i < game.rowActions.size(); i++) {
        // Выводим стратегию первого игрока.
        cout << game.rowActions[i] << "          ";

        // Выводим выигрыши для каждой пары стратегий.
        for (int j = 0; j < game.colActions.size(); j++) {
            cout << "("
                << game.matrix[i][j].first
                << ", "
                << game.matrix[i][j].second
                << ")"
                << "          ";
        }

        cout << endl;
    }

    cout << endl;
}

// Функция ищет строго доминируемые стратегии.
// Строго доминируемая стратегия всегда хуже другой стратегии
// при любых действиях второго игрока.
void findDominatedStrategies(const Game& game) {
    cout << "Анализ IESDA:\n";

    // Переменная показывает, найдена ли хотя бы одна доминируемая
    стратегия.
    bool foundDominated = false;

    // Количество стратегий первого игрока.
    int rows = game.rowActions.size();

    // Количество стратегий второго игрока.
    int cols = game.colActions.size();

    // Проверяем стратегии первого игрока.
    for (int x = 0; x < rows; x++) {
        // x - стратегия, которую проверяем.
        // y - другая стратегия, с которой сравниваем.
        for (int y = 0; y < rows; y++) {
            // Нельзя сравнивать стратегию саму с собой.
            if (x == y) {
                continue;
            }

            // Сначала считаем, что стратегия x доминируется
            стратегией y.
            bool dominated = true;

```

```

        // Проверяем выигрыш первого игрока при всех стратегиях
        второго игрока.
        for (int j = 0; j < cols; j++) {
            // Если стратегия y не даёт строго больший выигрыш,
            // значит стратегия x не является строго
            доминируемой.
            if (game.matrix[y][j].first <= game.matrix[x]
            [j].first) {
                dominated = false;
            }
        }

        // Если стратегия x строго доминируется стратегией y,
        // выводим результат.
        if (dominated) {
            foundDominated = true;

            cout << "Игрок 1: стратегия \""
                << game.rowActions[x]
                << "\" строго доминируется стратегией \""
                << game.rowActions[y]
                << "\".\n";
        }
    }
}

// Проверяем стратегии второго игрока.
for (int x = 0; x < cols; x++) {
    // x - стратегия второго игрока, которую проверяем.
    // y - другая стратегия второго игрока.
    for (int y = 0; y < cols; y++) {
        // Нельзя сравнивать стратегию саму с собой.
        if (x == y) {
            continue;
        }

        // Сначала считаем, что стратегия x доминируется
        стратегией y.
        bool dominated = true;

        // Проверяем выигрыш второго игрока при всех стратегиях
        первого игрока.
        for (int i = 0; i < rows; i++) {
            // Для второго игрока используем second.
            if (game.matrix[i][y].second <= game.matrix[i]
            [x].second) {
                dominated = false;
            }
        }

        // Если стратегия x строго доминируется стратегией y,

```

```

        // выводим результат.
        if (dominated) {
            foundDominated = true;

            cout << "Игрок 2: стратегия \""
                << game.colActions[x]
                << "\" строго доминируется стратегией \""
                << game.colActions[y]
                << "\".\n";
        }
    }

    // Если строго доминируемых стратегий нет, выводим сообщение.
    if (!foundDominated) {
        cout << "Строго доминируемых стратегий не найдено.\n";
    }

    cout << endl;
}

// Функция ищет равновесия Нэша в чистых стратегиях.
// Равновесие Нэша означает, что ни один игрок не может улучшить свой
выигрыш,
// если изменит только свою стратегию.
void findPureNashEquilibria(const Game& game) {
    cout << "Чистые равновесия Нэша:\n";

    // Переменная показывает, найдено ли хотя бы одно равновесие.
    bool found = false;

    // Количество стратегий первого игрока.
    int rows = game.rowActions.size();

    // Количество стратегий второго игрока.
    int cols = game.colActions.size();

    // Перебираем все клетки матрицы.
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            // Проверяем, является ли текущая стратегия первого
игрока
            // лучшим ответом на стратегию второго игрока.
            bool bestForPlayer1 = true;

            // Проверяем, является ли текущая стратегия второго
игрока
            // лучшим ответом на стратегию первого игрока.
            bool bestForPlayer2 = true;

            // Первый игрок может менять только строку.
            // Стратегия второго игрока остаётся неизменной.
            for (int otherRow = 0; otherRow < rows; otherRow++) {

```

```

        // Если первый игрок может получить больше,
        // выбрав другую строку, значит это не равновесие
Нэша.
        if (game.matrix[otherRow][j].first > game.matrix[i]
[j].first) {
            bestForPlayer1 = false;
        }
    }

    // Второй игрок может менять только столбец.
    // Стратегия первого игрока остаётся неизменной.
    for (int otherCol = 0; otherCol < cols; otherCol++) {
        // Если второй игрок может получить больше,
        // выбрав другой столбец, значит это не равновесие
Нэша.
        if (game.matrix[i][otherCol].second > game.matrix[i]
[j].second) {
            bestForPlayer2 = false;
        }
    }

    // Если оба игрока выбрали лучшие ответы друг на друга,
    // текущая клетка является равновесием Нэша.
    if (bestForPlayer1 && bestForPlayer2) {
        found = true;

        cout << "("
            << game.rowActions[i]
            << ", "
            << game.colActions[j]
            << ") ";

        cout << "выигрыш = ("
            << game.matrix[i][j].first
            << ", "
            << game.matrix[i][j].second
            << ")\n";
    }
}

// Если чистых равновесий Нэша нет, выводим сообщение.
if (!found) {
    cout << "Чистое равновесие Нэша не найдено.\n";

    // Для игры в монетку известно смешанное равновесие.
    if (game.name == "Игра в монетку") {
        cout << "Смешанное равновесие: каждый игрок
выбирает \"Орёл\" и \"Решка\" с вероятностью 0.5.\n";
    }
}

```

```

        cout << endl;
    }

    // Функция полностью анализирует одну игру.
    void analyzeGame(const Game& game) {
        // Выводим матрицу игры.
        printMatrix(game);

        // Ищем строго доминируемые стратегии.
        findDominatedStrategies(game);

        // Ищем чистые равновесия Нэша.
        findPureNashEquilibria(game);
    }

    int main() {
        // Настройка локали для русского текста.
        setlocale(LC_ALL, "");

#ifdef _WIN32
        // Настройка кодировки UTF-8 для Windows-консоли.
        SetConsoleOutputCP(CP_UTF8);
        SetConsoleCP(CP_UTF8);
#endif

        // Создаём первую игру: дилемма заключённого.
        Game prisonersDilemma;

        prisonersDilemma.name = "Дилемма заключённого";

        prisonersDilemma.situation =
            "Два заключённых выбирают: молчать или признаться. "
            "Каждый принимает решение независимо. "
            "Если один признаётся, а другой молчит, признавшийся получает лучший результат.";

        prisonersDilemma.rowActions = {"Молчать", "Признаться"};
        prisonersDilemma.colActions = {"Молчать", "Признаться"};

        // Матрица выигрышей:
        // оба молчат: (3, 3)
        // первый молчит, второй признаётся: (0, 4)
        // первый признаётся, второй молчит: (4, 0)
        // оба признаются: (1, 1)
        prisonersDilemma.matrix = {
            {{3, 3}, {0, 4}},
            {{4, 0}, {1, 1}}
        };

        // Создаём вторую игру: игра в монетку.
        Game matchingPennies;

        matchingPennies.name = "Игра в монетку";
    }

```

```

    matchingPennies.situation =
        "Два игрока одновременно выбирают сторону монеты: орёл или
    решка. "
        "Первый игрок выигрывает, если выборы совпали. "
        "Второй игрок выигрывает, если выборы различаются.";

    matchingPennies.rowActions = {"Орёл", "Решка"};
    matchingPennies.colActions = {"Орёл", "Решка"};

    // Матрица выигрышей:
    // если выборы совпали, первый игрок получает 1, второй получает
-1.
    // если выборы различаются, первый игрок получает -1, второй
получает 1.
    matchingPennies.matrix = {
        {{1, -1}, {-1, 1}},
        {{-1, 1}, {1, -1}}
    };

    // Создаём третью игру: игра на перекрёстке.
    Game crossroadGame;

    crossroadGame.name = "Игра на перекрёстке";

    crossroadGame.situation =
        "Два водителя подъезжают к перекрёстку. "
        "Каждый может ехать или остановиться. "
        "Если оба едут, возникает конфликт. "
        "Если один едет, а другой стоит, выигрывает тот, кто
    проехал.";

    crossroadGame.rowActions = {"Ехать", "Остановиться"};
    crossroadGame.colActions = {"Ехать", "Остановиться"};

    // Матрица выигрышей:
    // оба едут: (-1, -1)
    // первый едет, второй стоит: (1, 0)
    // первый стоит, второй едет: (0, 1)
    // оба стоят: (0, 0)
    crossroadGame.matrix = {
        {{-1, -1}, {1, 0}},
        {{0, 1}, {0, 0}}
    };

    // Анализируем все три игры.
    analyzeGame(prisonersDilemma);
    analyzeGame(matchingPennies);
    analyzeGame(crossroadGame);

    // Завершаем программу.
    return 0;
}

```